

Пояснительная записка

1. Наименование программы и краткое описание

В игре «Космическая оборона» (Кострыкиной Екатерины и Дмитрия Германа) вам предстоит расчислить путь для своего космического корабля от различных препятствий: комет, астероидов, метеоритов.

2. Краткая характеристика области применения, назначение разработки

Программа будет использоваться одной группой пользователей: игроками.

Игрокам будет представлено 5 уровней, за прохождение которых будут начисляться баллы, влияющие на финальный счёт игрока. Каждый уровень представляет собой тир с мишенями-целями, в которые нужно попасть определённое количество раз, чтобы «победить» их, также игроку необходимо уклоняться от ещё «живых» целей.

3. Описание работы программы

После запуска программы пользователю отображается начальное окно с расположенной на нём кнопками «Играть» и «Правила игры». После нажатия кнопки «Играть» игроку будет представлен меню уровней, которые будут открываться с прохождением предыдущих уровней. После нажатия кнопки «Правила игры» игроку будут представлены правила игры.

На уровне будут расположены цели с различными характеристиками:

- метеориты – обычные цели, со средней скоростью и степенью «живучести»;
- астероиды обладают средней скоростью и большой степенью «живучести»;
- кометы – высокая скорость, средняя степень «живучести»;
- враги-боссы – очень медленная скорость, высокая степень «живучести».

Также будут расположены спутники – объекты, в которые нельзя стрелять (за попадания по ним понижается счёт).

Игрок должен, управляя плазменной пушкой, убрать с пути своего космического корабля препятствия, не задевая спутники и уклоняясь от ещё «живых» целей.

После прохождения уровня на экран выведется финальное окно с итоговым счётом.

Реализованы классы:

- для всех целей Enemy, наследуемый от pygame.sprite.Sprite

[illegible]

- для комет Comet, наследуемый от Enemy;

```

637 class Comet(Enemy):
638     """Класс для комет"""
639     def __init__(self):
640         super().__init__()
641         self.health = 20
642         self.score = self.health
643         self.speed = 25
644         self.image = load_image("Picture/Comet.png")
645         self.rect = self.image.get_rect()
646         self.mask = pygame.mask.from_surface(self.image)
647

```

- для метеоритов Meteorite, наследуемый от Enemy;

```

593 class Meteorite(Enemy):
594     """Класс для метеоритов"""
595     def __init__(self):
596         super().__init__()
597         self.health = 10
598         self.score = self.health
599         self.speed = 35
600         self.image = pygame.transform.scale(load_image("Picture/Meteorite_1.png"), (80, 80))
601         self.rect = self.image.get_rect()
602         self.mask = pygame.mask.from_surface(self.image)
603

```

- для астероидов Asteroid, наследуемый от Enemy;

```

616 class Asteroid(Enemy):
617     """Класс для астероидов"""
618     def __init__(self):
619         super().__init__()
620         self.health = 30
621         self.score = self.health
622         self.speed = 20
623         self.image = load_image("Picture/Asteroid_1.png")
624         self.rect = self.image.get_rect()
625         self.mask = pygame.mask.from_surface(self.image)
626

```

- для спутников Satellite, наследуемый от Enemy;

```

652 class Satellite(Enemy):
653     """Класс для спутников"""
654     def __init__(self):
655         super().__init__()
656         self.health = 10
657         self.score = -60
658         self.speed = 15
659         self.image = pygame.transform.scale(load_image("Picture/Satellite.png"), (80, 80))
660         self.rect = self.image.get_rect()
661         self.mask = pygame.mask.from_surface(self.image)
662

```

- для врагов-боссов MeteoriteBoss и AsteroidBoss, наследуемые от Enemy;

```

605 class MeteoriteBoss(Enemy):
606     """Класс для метеоритов-боссов"""
607     def __init__(self):
608         super().__init__()
609         self.health = 100
610         self.score = self.health
611         self.speed = 15
612         self.image = pygame.transform.scale(load_image("Picture/Meteorite_2.png"), (150, 150))
613         self.rect = self.image.get_rect()
614         self.mask = pygame.mask.from_surface(self.image)

```

```

629 class AsteroidBoss(Enemy):
630     """Класс для астероидов-боссов"""
631     def __init__(self):
632         super().__init__()
633         self.health = 150
634         self.score = self.health
635         self.speed = 15
636         self.image = pygame.transform.scale(load_image("Picture/Asteroid_2.png"), (150, 150))
637         self.rect = self.image.get_rect()
638         self.mask = pygame.mask.from_surface(self.image)

```

- и др.

Основные функции, которые реализованы:

- `load_image()` – для загрузки изображений;

```

31 def load_image(name, colorkey=None):
32     """Функция для загрузки изображений"""
33     fullname = os.path.join('data', name)
34     if not os.path.isfile(fullname):
35         print(f"Файл с изображением '{fullname}' не найден")
36         sys.exit()
37     image = pygame.image.load(fullname)
38     if colorkey is not None:
39         image = image.convert()
40         if colorkey == -1:
41             colorkey = image.get_at((0, 0))
42             image.set_colorkey(colorkey)
43     else:
44         image = image.convert_alpha()
45     return image

```

- `cut_sheet()` – для деления изображения на фреймы и их последующего анимирования;

```

512 def cut_sheet(self, sheet, columns, rows):
513     """Функция для деления изображения на фреймы и их последующего анимирования"""
514     self.rect = pygame.Rect(0, 0, sheet.get_width() // columns,
515                             sheet.get_height() // rows)
516     for j in range(rows):
517         for i in range(columns):
518             frame_location = (self.rect.w * i, self.rect.h * j)
519             self.frames.append(sheet.subsurface(pygame.Rect(
520                 frame_location, self.rect.size)))

```

- `start_screen()`, `level_screen()`, `level_1()` и т. д. – для отображения отдельных окон

```

83 def start_screen():
84     """Функция для отображения и работы стартового окна"""
85     fon = pygame.transform.scale(load_image(f'Picture/SpaceBackground1.png'), (WIDTH, HEIGHT))
86     screen.blit(fon, (0, 0))
87
88     main_text = "КОСМИЧЕСКАЯ ОБОРОНА"
89     font = pygame.font.SysFont('MathSansBold', 35)
90     text = font.render(main_text, True, pygame.Color(252, 242, 255))
91     m_rect = text.get_rect()
92     m_rect.top = 210
93     m_rect.x = WIDTH // 2 - text.get_width() // 2
94     pygame.draw.rect(screen, (97, 69, 107), (m_rect.x - 10, m_rect.y - 10,
95     m_rect.width + 20, m_rect.height + 20))
96     screen.blit(text, m_rect)
97
98     text_coord = 345
99     x = 0
100     h = 0
101     w = 0
102     text_list = ["Правила игры", "Играть"]
103     font = pygame.font.SysFont('MathSans', 27)
104     for line in text_list:
105         text = font.render(line, True, pygame.Color(97, 69, 107))

```

```

134 def level_screen():
135     """Функция для отображения и работы окна с уровнями"""
136     fon = pygame.transform.scale(load_image(f'Picture/SpaceBackground2.png'), (WIDTH, HEIGHT))
137     screen.blit(fon, (0, 0))
138     pygame.draw.rect(screen, (252, 242, 255), (10, 10,
139     40, 40))
140     pygame.draw.polygon(screen, (97, 69, 107), [(20, 30), (35, 40), (35, 20)], 3)
141     font = pygame.font.SysFont('MathSansBold', 30)
142     text = font.render(f'СЧЕТ: {SCORE}', True, pygame.Color(252, 242, 255))
143     screen.blit(text, (215, 25))
144
145     text_list = ["Уровень 1", "Уровень 2", "Уровень 3", "Уровень 4",
146     "Уровень 5", "Скоро...", "Скоро...", "Скоро...", "Скоро..."]
147     font = pygame.font.SysFont('MathSansBold', 27)
148     text_y = 140
149     y = 100
150     k = 0
151     for i in range(3):
152         x = 60
153         text_x = 73

```

- и др.

4. Требования к составу и параметрам технических средств

Компьютер пользователя, включающий в себя:

- процессор x86 с тактовой частотой, не менее 1 ГГц;
- оперативную память объемом, не менее 1 Гб;
- видеокарту, монитор, мышь, клавиатура.

5. Список используемой литературы

1. ГОСТ 19.201-78 Единая система программной документации. Техническое задание. Требования к содержанию и оформлению. 1978. Режим доступа: <http://protect.gost.ru/document.aspx?control=7&id=155153>
2. Пример технического задания по ГОСТ 19.201-78. Режим доступа: <https://prof.com/forums/topic/%D0%BF%D1%80%D0%B8%D0%BC%D0%B5%D1%80-%D1%82%D0%B5%D1%85%D0%BD%D0%B8%D1%87%D0%B5%D1%81%D0%BA%D0%BE%D0%B3%D0%BE-%D0%B7%D0%B0%D0%B4%D0%B0%D0%BD%D0%B8%D1%8F-%D0%BF%D0%BE-%D0%B3%D0%BE%D1%81%D1%82-19-201-78>

3. Генератор космически фонов. Режим доступа: <https://deep-fold.itch.io/space-background-generator>
4. Анимированные изображения выстрелов. Режим доступа: <https://ansimuz.itch.io/warped-shooting-fx?download>
5. Звуки. Режим доступа: <https://kronbits.itch.io/freesfx?download>
6. Добавляем звук в игровой процесс. Модули mixer и music. Режим доступа: <https://proproprogs.ru/modules/dobavlyaem-zvuk-v-igrovoy-process-moduli-mixer-i-music>